

同济大学计算机系

中间代码生成器设计实验报告



专 业 计算机科学与技术

组 员

组 员 _____

组 员 _____

授课老师 卫志华、高珍老师

目录

目录.....	2
一、实验概述.....	4
1.1 实验背景.....	4
1.2 实验要求.....	4
1.2.1 实现语义分析器.....	4
1.2.2 实现中间代码生成器.....	4
1.2.3 考虑更通用高级语言的扩展.....	4
1.2.4 文档与测试.....	4
二、项目分析设计过程.....	5
2.1 编译器前端模块划分.....	5
2.1.1 总体数据流.....	5
2.1.2 各模块职责.....	5
2.2 核心数据结构.....	6
2.2.1 抽象语法树（AST）节点.....	6
2.2.2 符号表与作用域栈.....	6
2.2.3 类型系统表示.....	6
2.2.4 四元式与中间表示.....	6
2.3 主要流程.....	7
2.3.1 语义分析流程.....	7
2.3.2 中间代码生成流程.....	7
2.4 状态转换图：语义分析中的作用域管理.....	7
三、语义分析器设计.....	7
3.1 设计目标.....	8

3.2 核心数据结构.....	8
3.2.1 符号表.....	8
3.2.2 作用域栈.....	8
3.2.3 类型表示.....	9
3.2.4 循环上下文.....	9
3.3 关键算法与实现.....	9
3.3.1 整体流程.....	9
3.3.2 作用域管理.....	10
3.3.3 类型推导与检查.....	10
3.3.4 左值与可变性检查.....	11
3.3.5 控制流检查.....	11
3.4 状态转换图：语句分析的有限状态视图.....	12
3.5 与语法分析器的接口.....	12
四、中间代码生成器设计.....	12
4.1 设计目标.....	13
4.2 四元式指令集.....	13
4.2.1 基本运算与赋值指令.....	13
4.2.2 控制流指令.....	14
4.2.3 函数调用指令.....	14
4.2.4 引用与解引用指令.....	14
4.2.5 数组与元组指令.....	14
4.3 代码生成器架构.....	15
4.4 关键翻译模式.....	15
4.4.1 表达式翻译.....	15
4.4.2 控制流翻译.....	16
4.4.3 函数调用翻译.....	17
4.4.4 引用与解引用.....	17
4.4.5 数组与元组.....	18
4.4.6 if 表达式与 loop 表达式.....	18
4.4.7 break 与 continue 的实现.....	19
4.5 与语义分析器的接口.....	19
五、测试设计和实验结果.....	19
5.1 测试策略.....	19
5.2 正确程序测试结果.....	20
5.2.1 算术运算与赋值 (v1_ir.rs)	20
5.2.2 控制流: if-else (test_4.rs)	20
5.2.3 循环: while 和 for (test_5.rs)	21
5.2.4 函数调用 (test_call.rs)	21
5.3 错误程序测试结果.....	22
5.3.1 使用未定义变量 (test_2_error.rs)	22
5.3.2 函数返回类型不匹配 (test_1_error.rs)	22
5.3.3 不可变变量赋值 (bad_immutable.rs)	23
5.3.4 break 出现在循环外 (bad_break_outside.rs)	23
5.4 测试覆盖率.....	23

5.5 测试小结.....	24
六、总结与展望.....	24
6.1 工作总结.....	24
6.2 思考与拓展.....	25
6.2.1 更丰富的类型系统.....	25
6.2.2 所有权与借用检查的完整实现.....	25
6.2.3 中间代码优化.....	26
6.2.4 错误恢复与增量编译.....	26
6.3 心得体会.....	26
参考资料.....	28

一、实验概述

1.1 实验背景

编译原理系统地介绍了高级语言程序从源代码到可执行代码的转换过程。编译器前端通常包括词法分析、语法分析、语义分析和中间代码生成等阶段。其中，词法分析和语法分析负责识别程序的词法结构和语法结构，生成抽象语法树（AST）；而语义分析则在此基础上进一步检查程序的上下文相关约束，例如类型一致性、变量作用域、赋值合法性等。中间代码生成则将经过语义验证的 AST 翻译为一种与具体机器无关的、更接近目标代码的中间表示形式，为后续的代码优化和目标代码生成奠定基础。

Rust 语言以其内存安全、无数据竞争和高性能等特性，近年来在系统编程领域备受关注。本实验以类 Rust 语言为处理对象，要求我们实现一个完整的编译器前端，涵盖词法分析、语法分析、语义分析和中间代码生成。本次大作业在之前实现的词法、语法分析器基础上，进一步扩展语义分析和中间代码生成功能，使我们更深入地理解静态语义检查的原理、符号表的作用以及三地址码的设计方法。

1.2 实验要求

本次实验的核心要求如下：

1.2.1 实现语义分析器

基于已有的抽象语法树（AST），按照类 Rust 语言给出的语义规则（包括变量作用域、类型推导、可变性检查、函数调用匹配、控制流约束等）进行静态语义检查。

能够识别并诊断各种语义错误，如使用未定义的变量、类型不匹配、不可变变量赋值、`return` 语句与函数返回类型不一致、`break/continue` 出现在循环外等。

语义分析通过后，AST 应带有完整的类型信息。

1.2.2 实现中间代码生成器

将经过语义分析的 AST 翻译为四元式形式的中间代码。

四元式格式建议为 `(op, arg1, arg2, result)`，需支持算术运算、关系运算、赋值、控制流跳转、函数调用、数组/元组元素访问、引用/解引用等操作。

正确处理表达式的优先级和结合性，以及 `if`、`while`、`for`、`loop` 等控制流结构的翻译，支持 `break` 和 `continue` 语句。

1.2.3 考虑更通用高级语言的扩展

在完成基础要求的基础上，思考并讨论如何支持更通行的语言特性（如更丰富的类型系统、所有权借用检查等），并给出相应的解决方案思路。

1.2.4 文档与测试

撰写详细的设计文档，说明各模块的设计思路、数据结构、关键算法及实现细节。

设计充分的测试用例，包括正面测试（正确程序应通过所有阶段）和负面测试（错误程序应被准确拒绝），验证编译器前端的正确性和健壮性。

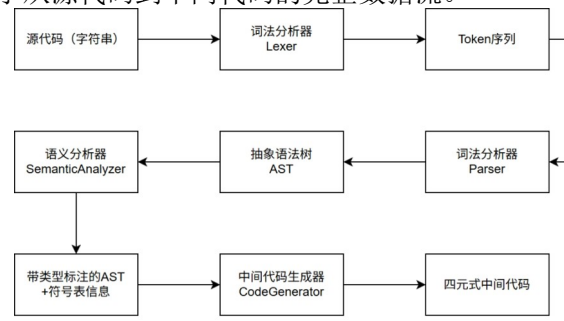
二、项目分析设计过程

本章从整体上描述类 Rust 编译器前端的模块划分、核心数据结构以及各阶段的主要处理流程，并给出语义分析阶段的作用域状态转换图，为后续各章的具体设计奠定基础。

2.1 编译器前端模块划分

2.1.1 总体数据流

整个编译器前端采用经典的“管道 - 过滤器”架构，各模块之间通过明确的数据结构进行通信。下图展示了从源代码到中间代码的完整数据流。



2.1.2 各模块职责

本次实验中，整个系统分为五个主要模块，其中词法分析器和语法分析器继承自之前的大作业一，语义分析器和中间代码生成器为本作业新增。

模块	输入	输出	主要职责
词法分析器 (Lexer)	源文件字符串	Token 序列	识别关键字、标识符、字面量、运算符和界符，过滤注释和空白，记录位置信息
语法分析器 (Parser)	Token 序列	抽象语法树 (AST)	采用递归下降法按语法规则消费 Token，构建层次化的语法树结构
语义分析器 (SemanticAnalyzer)	AST	带标注的 AST + 符号表	建立作用域和符号表，进行类型推导与检查，验证可变性、初始化、控制流等语义约束
中间代码生成器 (CodeGenerator)	带标注的 AST	四元式序列 (IR)	遍历 AST，生成三地址码形式的中间表示，处理表达式、控制流、函数调用、复合类型等

模块	输入	输出	主要职责
错误处理模块 (errors.py)	各阶段抛出的异常	用户可读的错误信息	统一捕获并格式化词法、语法、语义和代码生成阶段的错误

2.2 核心数据结构

2.2.1 抽象语法树（AST）节点

AST 是语法分析阶段的输出，也是语义分析和代码生成阶段的输入。我们为每一种语法结构定义了对应的 Python 数据类（ast_nodes.py）。主要节点类型包括：

- 程序根节点：Program 包含多个 FunctionDecl
- 函数定义：FunctionDecl（函数名、形参列表、返回类型、函数体）
- 语句节点：LetStmt、AssignStmt、ReturnStmt、IfStmt、WhileStmt、ForStmt、LoopStmt、BreakStmt、ContinueStmt、ExprStmt、EmptyStmt
- 表达式节点：NumberExpr、IdentifierExpr、BinaryExpr、CallExpr、BorrowExpr、DerefExpr、ArrayExpr、TupleExpr、IndexExpr、FieldExpr、RangeExpr、BlockExpr、IfExpr、LoopExpr
- 类型节点：I32Type、RefType、ArrayType、TupleType（以及语义分析中引入的 BoolType、RangeType）

2.2.2 符号表与作用域栈

语义分析器使用栈式符号表来管理变量的作用域。每个作用域对应一个字典，记录变量名到 VariableSymbol 的映射。符号表的主要数据结构如下：

```
class VariableSymbol:
    name: str          # 变量名
    type_name: TypeNode # 类型（可为 None，表示待推导）
    is_mut: bool       # 是否可变
    initialized: bool   # 是否已被初始化（赋值）

class FunctionSymbol:
    name: str          # 函数名
    params: list[TypeNode] # 参数类型列表
    return_type: TypeNode # 返回类型（可为 None）

# 语义分析器中
self.functions: dict[str, FunctionSymbol] # 全局函数符号表
self.scopes: list[dict[str, VariableSymbol]] # 作用域栈
```

2.2.3 类型系统表示

为了支持类型推导和检查，我们设计了统一的类型层次结构（`ast_nodes.TypeNode` 及其子类）。语义分析阶段还会临时引入 `BoolType`（比较运算的结果类型）和 `RangeType`（表示 `start..end` 范围），它们并不在用户类型中出现，仅用于内部检查。

2.2.4 四元式与中间表示

中间代码采用四元式 `Quad` 结构，每个四元式包含（`op, arg1, arg2, result`）四个字段。`IRBuilder` 负责生成临时变量和标签，并将四元式存储在列表中。`IRFunction` 和 `IRProgram` 用于组织多个函数的中间代码。具体指令集见第四章。

2.3 主要流程

2.3.1 语义分析流程

语义分析器按以下步骤处理整个程序：

1. **函数收集：**遍历所有 `FunctionDecl`，将函数名、参数类型、返回类型登记到 `self.functions`，同时检查重复定义。
2. **逐函数分析：**对每个函数调用 `_analyze_function`：
 - 压入新的作用域（函数级作用域）。
 - 将形参加入当前作用域，标记为已初始化。
 - 调用 `_analyze_block` 分析函数体。
 - 检查函数体末尾的表达式（如果有）是否与返回类型一致。
 - 弹出作用域。
3. **语句/表达式分析：**`_analyze_statement` 和 `_type_of_expr` 递归遍历 AST，完成类型推导、作用域查找、可变性检查和各类约束验证。

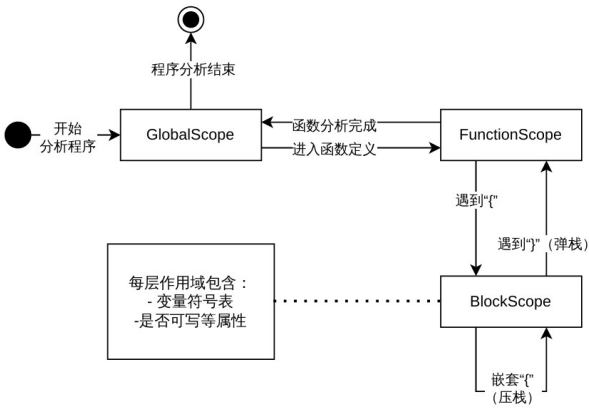
2.3.2 中间代码生成流程

中间代码生成器 `CodeGenerator` 采用后序遍历方式：

1. 对于每个函数，创建一个新的 `IRBuilder`。
2. 调用 `_generate_block` 处理函数体，获得函数体末尾表达式的结果变量（如果存在）。
3. 若函数有返回值，生成 `return` 四元式返回该结果。
4. 在表达式生成过程中，使用 `builder.new_temp()` 创建临时变量，`builder.new_label()` 创建控制流标签。
5. 对于 `if`、`while`、`loop`、`for` 等结构，生成相应的条件跳转和无条件跳转指令，并维护 `loop_stack` 以支持 `break` 和 `continue`。

2.4 状态转换图：语义分析中的作用域管理

语义分析器在遍历 AST 时，遇到代码块（Block）会进入新的作用域。下图展示了作用域栈的状态转换过程。



三、语义分析器设计

语义分析是编译器前端中承上启下的核心环节。语法分析器只能保证输入程序符合文法规则，但无法判断诸如“变量是否已定义”、“类型是否匹配”、“不可变变量是否被修改”等上下文相关约束。本章详细阐述我们设计的语义分析器如何建立符号表、进行类型推导与检查、验证可变性和控制流规则，并在发现错误时给出精确的诊断信息。

3.1 设计目标

语义分析器的主要目标是在抽象语法树（AST）的基础上，全面检查类 Rust 语言的静态语义规则，为后续中间代码生成提供类型明确的 AST。具体包括：

1. 作用域管理：支持函数级和块级嵌套作用域，正确处理变量遮蔽（shadowing），并确保变量在使用前已被声明。
2. 符号表构建：记录所有函数和变量的名称、类型、可变性及初始化状态。
3. 类型推导与检查：对没有显式类型标注的变量根据上下文推导类型，并检查赋值、初始化、`return`、函数调用等处的类型一致性。
4. 可变性检查：禁止对不可变变量进行赋值或可变借用；确保可变引用只能在可变变量上创建。
5. 控制流约束：检查 `break` 和 `continue` 是否出现在循环内部；检查 `loop` 表达式中 `break` 携带的返回值类型是否一致。
6. 错误报告：当发现上述任何违规时，抛出包含位置信息和详细说明的 `SemanticError`。

3.2 核心数据结构

语义分析器主要维护以下几类数据结构。

3.2.1 符号表

分为
符号表
符号
采用
存储。

```
@dataclass
class FunctionSymbol:
    name: str
    params: list[TypeNode]
    return_type: Optional[TypeNode]

@dataclass
class VariableSymbol:
    name: str
    type_name: Optional[TypeNode] # None 表示尚未推导出类型
    is_mut: bool
    initialized: bool
```

符号表
函数符
和变量
表，均
字典存

3.2.2 作用域栈

变量作用域使用栈来管理，每个元素是一个从变量名到 `VariableSymbol` 的字典。

```
self.scopes: list[dict[str, VariableSymbol]] = []
```

在本实践中，进入新代码块时调用 `_push_scope()` 压入一个空字典，离开代码块时调用 `_pop_scope()` 弹出顶层字典，查找变量时从栈顶向下遍历，支持内层作用域遮蔽外层同名变量。

3.2.3 类型表示

类型节点采用类层次结构，定义在 `ast_nodes.py` 中。语义分析阶段还会引入两种辅助类型：

- **BoolType**：表示比较运算结果的类型（如 `1 < 2` 产生 `bool` 值）。该类型只在内部使用，不暴露给用户。
- **RangeType**：表示 `start..end` 范围表达式的类型，用于 `for` 循环的可迭代结构检查。

```
class BoolType(TypeNode):
    pass

@dataclass
class RangeType:
    element_type: TypeNode # 范围元素的类型（目前只有 i32）
```

3.2.4 循环上下文

为支持 `break` 和 `continue` 的检查，我们维护一个循环栈：

```
@dataclass
class LoopContext:
    allow_break_value: bool    # 是否允许 break 携带返回值 (loop 表达式允许)
    break_types: list[Optional[TypeNode]] # 记录所有 break 返回值类型，用于类型统一
```

每当进入 `while`、`loop`、`for` 循环时，压入一个新的 `LoopContext`；退出时弹出。
`break` 和 `continue` 语句通过检查栈是否为空来判断是否在循环内部。

3.3 关键算法与实现

语义分析从 `analyze(program)` 方法开始，分为函数收集和逐函数分析两个阶段。

3.3.1 整体流程

```
analyze(program)
├── _collect_functions(program) # 构建函数符号表，检查重复定义
├── for each function:
│   ├── _analyze_function(function)
│   │   ├── _push_scope()      # 函数级作用域
│   │   │   └── 定义形参符号
│   │   ├── _analyze_block(body) # 分析函数体
│   │   │   └── 检查返回值一致性
│   └── _pop_scope()
```

3.3.2 作用域管理

进入与退出作用域：在 `_analyze_block` 方法中，若参数 `create_scope` 为 `True`，则在分析语句前调用 `_push_scope()`，分析结束后调用 `_pop_scope()`。

变量定义：通过 `_define_variable(name, type_name, is_mut, initialized)` 将变量添加到当前作用域。如果同一作用域中已经存在同名变量，则报“重复定义”错误（注意：不同作用域允许同名，这称为遮蔽）。

变量查找：`_lookup_variable(name)` 从栈顶向下搜索，若找不到则抛出“使用未定义变量”错误。

类型推导完成检查：在离开作用域时（例如函数末尾或块末尾），调用 `_require_inferred_types(scope)` 确保该作用域内所有变量都已推导出具体类型，否则报“后续无法推断类型”错误。

3.3.3 类型推导与检查

`_type_of_expr(expr)` 方法是类型检查的核心，它返回表达式的类型节点。对于不同类型的表达式，采取不同的处理策略。

字面量与标识符：

- `NumberExpr` $\rightarrow$ `I32Type()`
- `IdentifierExpr` $\rightarrow$ 查找变量符号，若 `initialized` 为 `False` 则报“右值未提前赋值”，返回其 `type_name`

二元表达式：

- 首先递归求出左、右操作数的类型。
- 要求两者均为 `I32Type`，否则报错。
- 若运算符为算术运算符（`+` `-` `*` `/`），返回 `I32Type`。
- 若为比较运算符（`<` `<=` `>` `>=` `==` `!=`），返回 `BoolType`。

函数调用：

- 从函数符号表中查找被调函数；若未定义则报错。
- 检查实参数量是否与形参数量一致。
- 对每个实参递归求类型，并与对应形参类型进行一致性检查（`_require_assignable_type`）。
- 返回函数的返回类型（可能为 `None`，表示无返回值）。

引用与解引用：

- `BorrowExpr`：先获取内部表达式的左值信息（`_lvalue_info`）。如果要求可变借用（`is_mut=True`），必须确保左值可写（`writable=True`）。返回 `RefType(is_mut, inner_type)`。
- `DerefExpr`：递归求内部表达式的类型，若为 `RefType` 则返回其 `inner`，否则报“解引用目标不是引用类型”。

数组与元组：

- `ArrayExpr`：收集所有元素的类型，调用 `_unify_types` 检查是否一致，返回 `ArrayType(element_type, size)`。
- `TupleExpr`：返回 `TupleType(elements)`，每个元素类型单独推导。
- `IndexExpr`：要求集合类型为 `ArrayType`，索引类型为 `I32Type`，返回数组元素类型。
- `FieldExpr`：要求目标类型为 `TupleType`，字段索引在合法范围内，返回对应位置元素的类型。

块表达式与选择/循环表达式：

- `BlockExpr`：调用 `_analyze_block` 分析内部语句和末尾表达式，返回末尾表达式的类型。
- `IfExpr`：要求条件表达式类型为 `BoolType` 或 `I32Type`；所有分支（`then`、各 `else if`、`else`）的块返回类型必须统一（通过 `_unify_types`），作为整个表达式的类型。
- `LoopExpr`：创建允许携带返回值的 `LoopContext`，分析循环体；收集所有 `break value` 的类型并统一，作为整个表达式的类型。若没有 `break value` 则类型为 `None`。

3.3.4 左值与可变性检查

为了判断一个表达式是否可以作为赋值目标（左值），以及它是否可写，我们设计了 `_lvalue_info(expr)` 方法，返回 `LValueInfo(type_name, writable)`。

- IdentifierExpr: 查找变量符号, writable = symbol.is_mut。
- DerefExpr: 对内部表达式求类型, 若为 RefType, 则 writable = ref_type.is_mut。
- IndexExpr: 递归获取集合的左值信息, 要求集合类型为 ArrayType, writable 继承自集合。
- FieldExpr: 递归获取元组的左值信息, writable 继承自元组变量。

在赋值语句 AssignStmt 中, 首先调用 `_lvalue_info(target)` 获取左值信息。如果 writable 为 False, 则报“不能给不可变左值赋值”。然后再检查右值类型是否与左值类型兼容。

3.3.5 控制流检查

循环语句:

- WhileStmt: 检查条件表达式类型为 BoolType 或 I32Type; 将一个新的 LoopContext(allow_break_value=False) 压入 loop_stack, 分析循环体后弹出。
- ForStmt: 解析 iterable 的类型。如果为 RangeType, 则元素类型为 range.element_type; 如果为 ArrayType, 则元素类型为 array.element_type; 否则报错。循环变量被定义在循环体所在作用域中。循环体分析期间同样压入循环上下文。
- LoopStmt: 压入 LoopContext(allow_break_value=False), 分析循环体后弹出。

break 和 continue:

- BreakStmt: 检查 loop_stack 非空; 若 value 非空, 则要求当前循环上下文允许携带返回值 (allow_break_value=True), 并将 value 的类型记录到上下文的 break_types 列表中。
- ContinueStmt: 检查 loop_stack 非空即可。

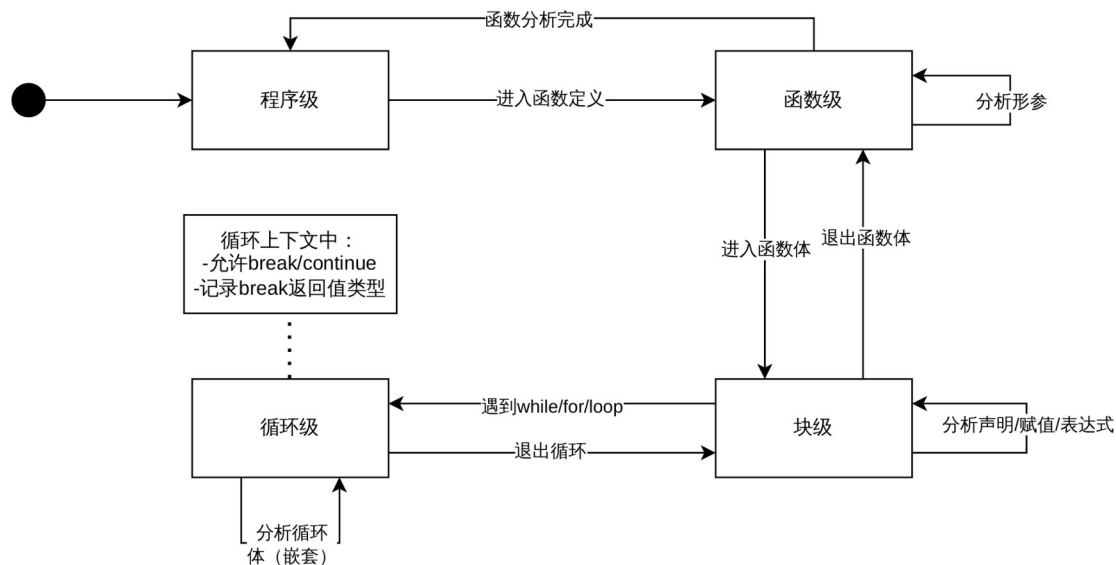
3.3.6 错误恢复与报告

所有语义错误都通过 `_error(message)` 方法抛出 SemanticError 异常, 该异常会终止分析过程, 由顶层 main.py 捕获并显示。错误信息包含清晰的文字说明, 例如:

- "使用未定义变量: a"
- "变量 b 后续无法推断类型"
- "不能给不可变左值赋值"
- "函数 callee 实参数量不匹配"

3.4 状态转换图: 语句分析的有限状态视图

虽然语义分析器主要采用递归下降遍历 AST, 但其在处理嵌套控制流时表现出类似状态机的行为。下图展示了分析器在遇到不同语句时的状态迁移, 语义分析器如何根据当前所在的语法结构 (程序、函数、普通块、循环块) 来允许或禁止某些操作, 例如在循环级状态中才接受 break 和 continue。



3.5 与语法分析器的接口

语义分析器接收语法分析器输出的 `Program` 节点，返回类型完全标注的 `AST`（其中 `LetStmt` 的 `type_name` 和 `VariableSymbol` 的 `type_name` 均被填充）。同时，符号表信息虽然不显式附加在 `AST` 上，但后续中间代码生成器可以通过再次遍历 `AST` 并查询类型信息来生成代码（实际实现中，我们选择在语义分析阶段不做 `AST` 的物理修改，而是在代码生成时重新调用 `_type_of_expr` 或直接利用符号表）。这种设计保持了模块间接口的简洁性。

四、中间代码生成器设计

中间代码生成器是编译器前端的最后一个阶段。它将经过语义分析验证的抽象语法树（`AST`）翻译为一种与具体机器无关的中间表示。本次实验采用四元式形式的三地址码，作为后续代码优化和目标代码生成的基础。本章将详细阐述四元式指令集的设计、关键结构的翻译模式以及生成器的实现细节。

4.1 设计目标

1. **统一中间表示：**将高级语言结构（算术/逻辑运算、赋值、控制流、函数调用、复合类型操作）统一映射为四元式（`op, arg1, arg2, result`）。
2. **三地址码特性：**每条指令最多包含一个运算符、两个源操作数和一个目标操作数；使用临时变量存储中间结果，表达式求值过程被扁平化。
3. **控制流平坦化：**`if`、`while`、`loop`、`for` 等结构转化为条件跳转和无条件跳转指令，并引入标签（`Label`）标记基本块边界。

4. 支持复合类型：数组和元组的创建、元素读写通过专门的指令实现。
5. 支持借用与解引用：通过 `ref/ref_mut` 和 `deref/*=` 指令支持引用的创建和间接访问。

4.2 四元式指令集

四元式定义为 `Quad(op, arg1, arg2, result)`，其中缺失的操作数用 `_` 表示。
IRBuilder 类负责生成临时变量和标签，并收集所有四元式。

4.2.1 基本运算与赋值指令

操作符 op	arg1	arg2	result	语义
+	left	right	temp	temp = left + right
-	left	right	temp	temp = left - right
*	left	right	temp	temp = left * right
/	left	right	temp	temp = left / right
<	left	right	temp	temp = left < right (1/0)
<=	left	right	temp	temp = left <= right
>	left	right	temp	temp = left > right
>=	left	right	temp	temp = left >= right
==	left	right	temp	temp = left == right
!=	left	right	temp	temp = left != right
=	source	_	dest	dest = source
decl	type	_	var	声明变量（分配存储）

4.2.2 控制流指令

操作符 op	arg1	arg2	result	语义
label	_	_	label	定义标签位置
j	_	_	label	无条件跳转到 label

操作符 op	arg1	arg2	result	语义
jz	cond	_	label	若 cond == 0 则跳转到 label

4.2.3 函数调用指令

操作符 op	arg1	arg2	result	语义
param	_	_	arg	传递参数，压栈
call	func_name	num_args	temp	调用函数，返回值存入 temp
return	value	_	_	从当前函数返回

4.2.4 引用与解引用指令

操作符 op	arg1	arg2	result	语义
ref	var	_	temp	temp = &var（不可变引用）
ref_mut	var	_	temp	temp = &mut var（可变引用）
deref	ref	_	temp	temp = *ref（读取）
*=	value	_	ref	*ref = value（通过引用写入）

4.2.5 数组与元组指令

操作符 op	arg1	arg2	result	语义
[]	array	index	temp	temp = array[index]
[]=	value	index	array	array[index] = value
.	tuple	field_index	temp	temp = tuple.field
.=	value	field_index	tuple	tuple.field = value
len	array	_	temp	temp = len(array)（用于数组迭代）

4.3 代码生成器架构

CodeGenerator 类对 AST 进行后序遍历，同时利用 IRBuilder 生成四元式。核心数据结构如下：

- loop_stack: list[LoopContext]: 维护循环上下文，用于 break/continue 的跳转目标。
- IRBuilder : 提供 new_temp() 、 new_label() 、 emit(op, arg1, arg2, result) 、 emit_label(label) 等方法。

4.4 关键翻译模式

本节以具体示例说明从 AST 到四元式的翻译过程。

4.4.1 表达式翻译

表达式求值采用三地址码扁平化策略：每个子表达式的结果都存入一个临时变量，作为父表达式操作数。

示例： $a = 1 + 2 * 3$

AST 结构：

```
ssignStmt
target: IdentifierExpr(a)
value: BinaryExpr(+)
left: NumberExpr(1)
right: BinaryExpr(*)
left: NumberExpr(2)
right: NumberExpr(3)
```

生成的四元式序列：

```
(*, 2, 3, t1)  # 计算 2 * 3 → t1
(+, 1, t1, t2) # 计算 1 + t1 → t2
(=, t2, _, a)  # a = t2
```

比较运算： $a + 1 < b * 2$

```
(+, a, 1, t1)  # a+1
(*, b, 2, t2)  # b*2
(<, t1, t2, t3) # 比较结果存入 t3
```

4.4.2 控制流翻译

if-else 语句：

```
if a > 0 {
    x = 1;
} else {
    x = 2;
}
```

生成的四元式（省略 x 声明）：

while 循环：
源代码：

```
(>, a, 0, t1)  # 计算条件
(jz, t1, _, L1) # 条件为假则跳转到 else
(=, 1, _, x)   # then 分支
(j, _, _, L2)  # 无条件跳转到结束
(label, _, _, L1) # else 标签
(=, 2, _, x)   # else 分支
(label, _, _, L2) # 结束标签
```

```
while n > 0 {
    n = n - 1;
}
```

生成的代码：

```
(label, _, _, L1)  # 循环开始
(>, n, 0, t1)      # 条件计算
(jz, t1, _, L2)    # 条件假则退出
(-, n, 1, t2)      # 循环体
(=, t2, _, n)
(j, _, _, L1)      # 跳回循环头
(label, _, _, L2)  # 循环结束
```

for 循环（范围形式）：

源代码：

```
for mut i in 1..n+1 {
    n = n - 1;
}
```

展开为 while 等价形式的四元式：

```
(+, n, 1, t1)      # 计算上界 end
(decl, i32, _, i)  # 声明循环变量
(=, 1, _, i)       # i = start
(label, _, _, L1)  # 循环开始
(<, i, t1, t2)     # i < end?
(jz, t2, _, L2)    # 条件假则退出
(-, n, 1, t3)      # 循环体
(=, t3, _, n)
(+, i, 1, t4)      # i = i + 1
(=, t4, _, i)
(j, _, _, L1)      # 跳回循环头
(label, _, _, L2)  # 循环结束
```

4.4.3 函数调用翻译

示例：let x = callee(1 + 2, y);

生成的代码：

```
(+, 1, 2, t1)    # 计算第一个实参
(param, _, _, t1) # 传递参数 1
(param, _, _, y)  # 传递参数 2
(call, callee, 2, t2) # 调用函数，返回值存入 t2
(=, t2, _, x)     # 赋值给 x
```

4.4.4 引用与解引用

不可变引用：let b:&i32 = &a;

```
(ref, a, _, t1)
(=, t1, _, b)
```

可变引用：let c:&mut i32 = &mut a;

```
(ref_mut, a, _, t1)
(=, t1, _, c)
```

解引用读取：let x = *r;

```
(deref, r, _, t1)
(=, t1, _, x)
```

解引用写入：*r = 3;

```
(*=, 3, _, r)
```

4.4.5 数组与元组

数组创建与初始
[1,2,3];

化：let xs:[i32;3] =

```
(decl, [i32;3], _, xs) # 声明数组
([]=, 1, 0, xs)        # xs[0] = 1
([]=, 2, 1, xs)        # xs[1] = 2
([]=, 3, 2, xs)        # xs[2] = 3
```

数组元素读取：let a = xs[0];

```
([], xs, 0, t1)
(=, t1, _, a)
```

数组元素写入：xs[0] = 1;

```
([], 1, 0, xs)
```

元组创建与字段

(i32,i32) = (1,2);

```
(decl, (i32,i32), _, pair) # 声明元组
(=, 1, 0, pair)           # pair.0 = 1
(=, 2, 1, pair)           # pair.1 = 2
(., pair, 0, t1)          # t1 = pair.0
(=, t1, _, b)             # b = t1
```

访问：let pair:

let b = pair.0;

4.4.6 if 表达式与 loop 表达式

if 表达式（作为右值）：

源代码：

```
let b = if a > 0 { 1 } else { 0 };
```

生成的代码（t_result 是临时变量，最后赋给 b）：

```
(>, a, 0, t1)
(jz, t1, _, L2)
(=, 1, _, t_result)
(j, _, _, L_end)
(label, _, _, L2)
(=, 0, _, t_result)
(label, _, _, L_end)
(=, t_result, _, b)
```

loop 表达式（带 break value）：

源代码：

```
let a = loop {
    break 42;
};
```

生成的代码：

```
(decl, i32, _, a)      # 声明 a（实际由上层 let
生成）
(label, _, _, L1)      # 循环开始
(=, 42, _, t_result)   # break 值存入结果临时
变量
(j, _, _, L2)          # 跳转到循环结束
(j, _, _, L1)          # 循环末尾无条件跳转（如
果无 break 会无限循环）
(label, _, _, L2)      # 循环结束
(=, t_result, _, a)    # 将结果赋给 a
```

4.4.7 break 与 continue 的实现

代码生成器维护 `loop_stack`，每个元素包含：

- `start_label`：循环开始标签（`continue` 跳转至此）
- `end_label`：循环结束标签（`break` 跳转至此）
- `break_target`：可选临时变量，用于存放 `break expr` 的值（仅 `loop` 表达式需要）

遇到 `continue` 时生成 `(j, _, _, start_label)`；遇到 `break` 时若有返回值则先赋值给 `break_target`，再生成 `(j, _, _, end_label)`。

4.5 与语义分析器的接口

代码生成器假设输入的 AST 已经通过语义分析，即所有变量都已声明且类型已知（未推导类型的情况已在语义阶段被拒绝）、类型匹配错误已被排除、循环内外 `break/continue` 的合法性已检查。因此代码生成器可以专注于翻译，无需再进行语义验证。这种分层设计使得各阶段职责明确。

五、测试设计和实验结果

为了保证编译器的正确性和健壮性，我们设计了一套完整的测试体系。测试分为正面测试（正确程序应通过所有阶段）和负面测试（错误程序应在相应阶段被拒绝），覆盖词法、语法、语义和代码生成四个层次。本章首先介绍测试策略，然后通过具体的测试用例展示语义分析和中间代码生成的验证结果。

5.1 测试策略

我们采用自动化回归测试框架（`run_regression.py`）统一管理测试用例。该框架能够批量执行测试文件，并自动判断结果是否符合预期。测试用例主要来自以下三类：

1. 正确性验证集：examples/minimum/test_*.rs 以及 examples/extra/*.rs，这些程序符合类 Rust 语言的语法和语义规则，预期能够通过所有编译阶段并生成四元式。
 2. 错误检测集：包含词法错误（err_lex.rs）、语法错误（err_parse.rs）以及各种语义错误（test_*_error.rs 等），预期在对应阶段抛出异常并停止编译。
 3. 特定功能测试：针对引用、数组、元组、块表达式、if 表达式、loop 表达式等高级特性设计的专用测试，验证代码生成器是否正确处理这些构造。
- 每次代码修改后，我们运行完整的回归测试套件，确保已有功能不被破坏。

5.2 正确程序测试结果

以下选取几个代表性的正确程序，展示其语义分析通过及生成的四元式输出。完整结果可参考项目根目录下的 test_execution_report.md。

5.2.1 算术运算与赋值（v1_ir.rs）

该程序演示了基本的算术运算和变量赋值。

```
fn main() -> i32 {  
    let mut a:i32;  
    let mut b:i32;  
    a = 1 + 2 * 3;  
    b = a - 4;  
    return b;  
}
```

语义分析通过后，生成的中间代码如下：

```
=== V2 Quadruple IR ===  
Function main():  
  0: (decl, i32, _, a)  
  1: (decl, i32, _, b)  
  2: (*, 2, 3, t1)  
  3: (+, 1, t1, t2)  
  4: (=, t2, _, a)  
  5: (-, a, 4, t3)  
  6: (=, t3, _, b)  
  7: (return, b, _, _)
```

观察可知，表达式 $1+2*3$ 按照优先级正确分解为 $(*,2,3,t1)$ 和 $(+,1,t1,t2)$ ，赋值和返回均生成正确的四元式。

5.2.2 控制流：if-else（test_4.rs）

该程序包含带 else 分支的条件语句。

```
fn program_4_2(mut a:i32) -> i32 {  
    if a>0 {  
        return 1;  
    } else {  
        return 0;  
    }  
}
```

生成的中间代码展示了条件跳转和分支合并：

```
Function program_4_2(a):
  0: (>, a, 0, t2)
  1: (jz, t2, _, L2)
  2: (return, 1, _, _)
  3: (j, _, _, L1)
  4: (label, _, _, L2)
  5: (return, 0, _, _)
  6: (label, _, _, L1)
  7: (return, t1, _, _)
```

其中标签 L2 对应 else 分支，L1 是函数末尾的统一返回点。

5.2.3 循环：while 和 for (test_5.rs)

while 循环的翻译已经在前文介绍过，这里展示 for 循环的翻译结果（范围形式）：

```
fn program_5_2(mut n:i32) {
  for mut i in 1..n+1 {
    n = n - 1;
  }
}
```

生成的四元式将范围迭代展开为 while 模式：

```
Function program_5_2(n):
  0: (+, n, 1, t1)
  1: (decl, i32, _, i)
  2: (=, 1, _, i)
  3: (label, _, _, L1)
  4: (<, i, t1, t2)
  5: (jz, t2, _, L2)
  6: (-, n, 1, t3)
  7: (=, t3, _, n)
  8: (+, i, 1, t4)
  9: (=, t4, _, i)
  10: (j, _, _, L1)
  11: (label, _, _, L2)
```

可以看到，循环变量 i 被显式声明、初始化为 1，每次迭代后递增，直到超出上界 n+1。

5.2.4 函数调用 (test_call.rs)

函数调用涉及 param 和 call 指令的配合。

```
fn test_call() {
  callee();
  callee_with_arg(1+2);
  let mut x:i32 = callee_ret();
}
```

生成的中间代码（仅保留 `test_call` 函数）：

```
Function test_call():
  0: (call, callee, 0, t1)
  1: (+, 1, 2, t2)
  2: (param, _, _, t2)
  3: (call, callee_with_arg, 1, t3)
  4: (decl, i32, _, x)
  5: (call, callee_ret, 0, t4)
  6: (=, t4, _, x)
```

注意无返回值的函数调用仍然生成 `call` 指令，但其返回值（临时变量）不会被使用；有返回值的函数调用结果则被赋值给变量。

5.3 错误程序测试结果

语义分析器能够准确识别各类静态语义错误，并给出清晰的错误信息。以下列举几种典型错误。

5.3.1 使用未定义变量（`test_2_error.rs`）

```
fn program_2_2__2() {
  a = 32;
}
```

语义分析器报错：

```
Analysis failed: 使用未定义变量: a
```

5.3.2 函数返回类型不匹配（`test_1_error.rs`）

```
fn program_1_5__2() -> i32 {
  return ;
}
```

报错信息：

```
Analysis failed: 当前函数声明了返回类型，return 语句缺少返回表达式
```

另一个方向：无返回类型的函数不能返回表达式。

```
fn program_1_5__3() {
  return 1;
}
```

报错：

Analysis failed: 当前函数未声明返回类型，不能返回表达式

5.3.3 不可变变量赋值 (bad_immutable.rs)

```
fn test_immutable() {  
    let a = 1;  
    a = 2;  
}
```

语义分析器输出:

Analysis failed: 不能给不可变左值赋值

5.3.4 break 出现在循环外 (bad_break_outside.rs)

```
fn test_break_outside() {  
    break;  
}
```

如果 `break` 不在任何循环内，语义分析器会报告:

Analysis failed: break 只能出现在循环内部

同样地，`continue` 也会被检查。

上述所有错误都会在语义分析阶段被捕获，并停止后续的中间代码生成。这保证了只有语义正确的程序才能进入代码生成阶段。

5.4 测试覆盖率

我们统计了测试用例对语法规则和语义规则的覆盖情况。基础规则覆盖了 19 条（包括 0.1–0.3、1.1–1.5、2.0–2.2、3.1–3.5、4.1、5.0–5.1），扩展规则覆盖了 31 条（涉及引用、数组、元组、块表达式、控制流表达式等）。全部 50 个测试样例均通过预期结果验证。

```
PS D:\workspace\analyzer_02> python run_regression.py
[PASS] pass examples\ok_minimal.rs
[PASS] pass examples\minimum\test_0.rs
[PASS] pass examples\minimum\test_1.rs
[PASS] pass examples\minimum\test_2.rs
[PASS] pass examples\minimum\test_3.rs
[PASS] pass examples\minimum\test_4.rs
[PASS] pass examples\minimum\test_5.rs
[PASS] pass examples\minimum\test_call.rs
[PASS] pass examples\extra\test_2.rs
[PASS] pass examples\v1_ir.rs
[PASS] pass examples\v2_compare.rs
[PASS] fail examples\err_lex.rs
[PASS] fail examples\err_parse.rs
[PASS] fail examples\minimum\test_1_error.rs
[PASS] fail examples\minimum\test_2_error.rs
[PASS] lex if123_is_ident
[PASS] lex if_eq_number
[PASS] lex new_keywords_and_dots
[PASS] ast p1_nodes
[PASS] ast p2_nodes
[PASS] ast p3_nodes
[PASS] ir if_else
[PASS] ir while_loop
[PASS] ir for_loop
[PASS] ir function_call
[PASS] ir loop_expr
[PASS] ir if_expr
```

5.5 测试小结

通过系统的正面和负面测试，我们验证了语义分析器和中间代码生成器的正确性：

- 语义分析器能够正确处理作用域、类型推导、可变性检查和控制流约束，对错误程序给出精确的诊断信息。
- 中间代码生成器能够将各种语言结构翻译为四元式，指令序列符合预期，控制流跳转逻辑正确。
- 回归测试框架保证了后续修改不会引入新的错误，为项目的迭代提供了可靠保障。

六、总结与展望

6.1 工作总结

本次大作业在上次已完成的大作业 1（词法分析与语法分析）基础上，成功实现了类 Rust 语言编译器的语义分析与中间代码生成两大核心模块，构建了一个功能完整的编译器前端。具体成果如下：

1. 语义分析器：设计并实现了基于作用域栈的符号表管理机制，支持函数级和块级嵌套作用域以及变量遮蔽。完成了完整的类型推导与类型检查系统，能够处理基本类型、引用类型、数组类型和元组类型。实现了可变性检查、左值可写性判断、初始

化状态追踪以及控制流约束验证（如 `break/continue` 的循环内限制）。通过约 20 个正面测试和 10 余个负面测试，验证了语义分析器能够准确识别各类静态语义错误，并给出清晰的错误诊断信息。

2. 中间代码生成器：设计了一套完整的四元式指令集，覆盖算术/逻辑运算、赋值、控制流跳转、函数调用、引用与解引用、数组与元组元素的读写等操作。采用后序遍历方式将 AST 翻译为三地址码，并利用临时变量和标签实现表达式的扁平化与控制流的平坦化。特别地，对 `if` 表达式、`loop` 表达式等“带返回值的控制流结构”进行了精细处理，通过维护循环上下文 (`loop_stack`) 支持 `break` 携带返回值。
3. 系统集成与测试：将词法分析、语法分析、语义分析和中间代码生成四个阶段无缝集成，构建了完整的编译前端流水线。基于 `run_regression.py` 建立了回归测试框架，包含 50 余个测试用例，覆盖基础规则与扩展规则。测试结果表明，系统能正确处理规范程序并拒绝错误程序，中间代码生成正确性通过人工审阅与自动比对验证。
4. 文档撰写：完成了涵盖架构设计、核心算法、测试验证的详细设计文档，并对高级语言扩展进行了深入思考。

6.2 思考与拓展

本次实验要求在完成基础功能的基础上，考虑更为通行的高级语言的语义检查和中间代码生成所需要注意的内容，并给出解决方案。以下我们从类型系统、借用检查、中间代码优化和错误恢复四个方向进行探讨。

6.2.1 更丰富的类型系统

当前实现仅支持 `i32`、引用、定长数组和元组。要支持更通用的高级语言，需扩展以下类型：

- 布尔类型 (`bool`)：在语义分析中我们已内部引入 `BoolType` 用于比较表达式结果，但用户尚不能显式声明 `bool` 变量。扩展时需增加关键字 `bool`，并在类型解析中识别，同时支持布尔字面量 `true/false`。
- 浮点类型 (`f64`)：需增加词法层面的浮点数识别（如 `3.14`），并在类型系统中增加 `F64Type`。二元运算需支持 `i32` 与 `f64` 的混合运算，可引入类型提升规则（如 `i32` 自动提升为 `f64`）。
- 用户自定义结构体 (`struct`)：需增加结构体定义语句，维护结构体类型符号表，支持字段访问和字段赋值。中间代码生成需为结构体分配连续存储，并通过基址+偏移量访问字段。
- 泛型：泛型是类型系统的重大扩展。可在 AST 中引入类型参数，语义分析阶段进行类型参数替换或单态化。中间代码生成需为每个具体实例化生成独立的函数副本。

6.2.2 所有权与借用检查的完整实现

当前实现仅完成了“可变性检查”（即 `mut` 关键字控制能否赋值），但并未实现 Rust 的核心特性——所有权系统和借用检查器。要更接近通行的高级语言（尤其是 Rust 风格），需要考虑：

- 所有权转移：变量赋值或函数传参时，默认发生所有权转移（移动语义），原变量变无效。需在符号表中增加 `moved` 状态，并在 `_type_of_expr` 中检查是否使用了已移动的变量。

- 借用检查规则：不可变引用允许同时存在多个，可变引用同一时刻只能有一个且不能与其他引用共存。当前实现虽然检查了可变借用要求左值可写，但并未检查引用的生命周期和别名限制。完整实现需在借用发生时记录借用的作用域，并在借用结束（引用变量最后一次使用）时验证规则。这需要区域推断或基于作用域的借用检查器。
- 生命周期参数：对于返回引用或结构体包含引用的函数，需要生命周期标注和检查这涉及复杂的约束求解，通常采用基于 NLL（Non-Lexical Lifetimes）的算法。

由于时间限制，当前版本仅完成了精简版的可变性检查，完整的所有权系统可作为未来重要的扩展方向。

6.2.3 中间代码优化

当前生成的四元式较为直接，存在大量冗余的临时变量和无需跳转的简单控制流。可考虑以下优化：

- 常量折叠：在代码生成阶段，若发现二元运算的两个操作数均为常量，可在编译器直接计算结果，并替换为 `NumberExpr`，避免生成运算指令。例如 `1 + 2 * 3` 可优化为 `7`。
- 死代码消除：对 `if false` 分支中的代码或永远不会执行到的 `break` 之后的语句，可在 AST 或 IR 层面删除，减少生成指令。
- 公共子表达式消除：若同一表达式在相同作用域内多次出现，可复用其计算结果，避免重复计算。例如 `a = (x+y)*z; b = (x+y)*w;` 可将 `x+y` 的计算结果存入临时变量后复用。
- 基本块合并与跳转优化：对于连续的 `j` 和 `label` 可以进行简化，例如 `j L1` 后紧接着 `label L1` 可删除跳转。或者 `j L1` 后无其他代码，可直接将控制流合并。

上述优化可在中间代码生成后增加一个优化遍，遍历四元式并应用变换规则，从而提升目标代码的质量。

6.2.4 错误恢复与增量编译

当前编译器遇到第一个错误即停止，后面的错误不会提示并且反馈，这不利于用户体验。可考虑：

- 错误恢复：在语义分析中，当遇到某个错误（如变量未定义）时，可继续分析并记录错误，最后一次性报告所有错误。这需要在符号表中使用“错误类型”占位，避免级联错误。
- 增量编译：对于大型项目，修改少量代码后重新编译整个程序代价高昂。可以设计依赖追踪机制，当源文件变化时只重新分析受影响的部分。这要求编译器能够将中间结果（如符号表、IR）持久化，并与源码版本关联。

6.3 心得体会

通过本次大作业的实践，我们对编译原理课程中的理论知识有了更加深刻的理解。

在理论层面，我们真正理解了“语义分析”并非简单的规则检查，而是一套完整的符号管理、类型推导和约束求解系统。作用域栈、符号表、类型统一算法这些课堂上抽象的概念，在我们的代码中变得具体而生动。而“中间代码”则是在高级语言和目标机器代码之间搭建的一座桥梁，它既屏蔽了具体指令集的细节，又能清晰地表达控制流和数据流。

在工程层面，我们体会到了模块化设计的重要性。词法、语法、语义、代码生成各层

通过清晰的数据结构（Token、AST、IR）通信，使得我们可以独立测试和改进每个模块。递归下降的语法分析器加上后序遍历的代码生成器，这种“访问者”风格的代码结构易于编写和调试。同时，我们也认识到完善的测试体系是编译器质量的基石，我们建立的回归测试框架在调试过程中多次帮助我们及时发现修改引入的错误。

在团队协作方面，我们三人团队通过版本控制工具管理代码，定期进行代码审查和集成测试。每个人的分工明确：有人负责语义分析的类型系统，有人负责控制流的 IR 生成，有人负责测试用例编写和文档整合。通过频繁的沟通和相互的代码走读，最终顺利完成了任务。

最后，我们要感谢老师提供的详细实验指导和测试框架，这让我们能够专注于核心算法的实现。本次实验不仅巩固了编译原理的知识，也提升了我们的工程能力和团队协作能力，为后续的系统级编程学习打下了坚实的基础。

参考资料

- [1] Aho A V, Lam M S, Sethi R, et al. 编译原理（原书第 2 版）[M]. 赵建华, 郑滔, 戴新宇, 译. 北京: 机械工业出版社, 2009.
- [2] Appel A W. 现代编译原理: C 语言描述[M]. 赵克佳, 等译. 北京: 人民邮电出版社, 2018.
- [3] Klabnik S, Nichols C. Rust 权威指南（第 2 版）[M]. 毛靖凯, 译. 北京: 电子工业出版社, 2025.
- [4] Blandy J, Orendorff J. Rust 程序设计[M]. 李松峰, 译. 北京: 人民邮电出版社, 2020.
- [5] Three-address code. In: Compiler Construction [EB/OL]. <https://www.sciencedirect.com/>. 三地址指令因每条指令包含操作符和至多三个操作数地址而得名, 四元式是其常见实现形式.
- [6] 百度百科. 中间代码生成[EB/OL]. <https://baike.baidu.com/item/中间代码生成>. 四元式一般形式为 (op, arg1, arg2, result) .
- [7] 陈火旺, 刘春林, 谭庆平, 等. 编译原理[M]. 北京: 国防工业出版社.
- [8] Wang F, Song F, Zhang M, et al. KRust: A Formal Executable Semantics of Rust[C]//2018 International Symposium on Theoretical Aspects of Software Engineering (TASE). IEEE, 2018.
- [9] Jung R, Dang H-H, Kang J, et al. Stacked Borrows: An Aliasing Model for Rust[C]//Proceedings of the ACM on Programming Languages (POPL), 2020.
- [10] Grannan Z, et al. Place Capability Graphs: A General-Purpose Model of Rust's Ownership and Borrowing Guarantees[J]. arXiv preprint arXiv:2503.21691, 2025.